CS 4375 HW3: Neural Network Report

Shivesh Gupta
NetID: SXG220223

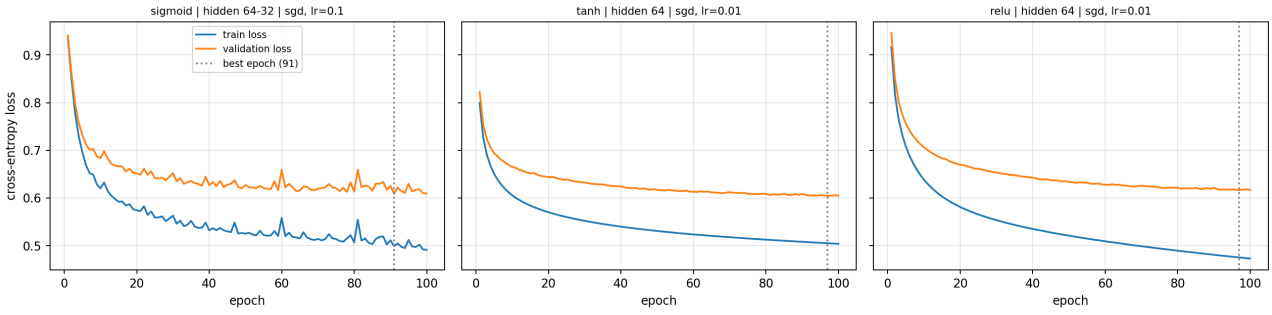
October 2026

Setup

I used the UCI *Predict Students' Dropout and Academic Success* dataset (`https://archive.ics.uci.edu/dataset/`
It has 4,424 students, 36 features and 3 classes: Graduate (49.9%), Dropout (32.1%) and Enrolled (17.9%). The split is 70/15/15, stratified (3096 / 664 / 664), with seed 42. After one-hot encoding the 10 categorical columns and standardizing the rest, there are 234 inputs. The output layer is softmax with cross-entropy. Because the classes are imbalanced I report macro-F1 too and used validation macro-F1 to pick the best configuration.

I ran 24 configurations: 3 activations × 2 architectures (64 and 64–32) × SGD (lr 0.1, 0.01) or Adam (lr 0.01, 0.001). Training stops early when the validation loss has not improved for 15 epochs (max 100 epochs) and the best weights are kept. The full table is in `results.pdf`.

Activation	Hidden	Optimizer (lr)	Train acc	Val acc	Val F1	Test acc	Test F1
Sigmoid	64–32	SGD (0.1)	80.17	77.11	70.99	79.67	73.09
Tanh	64	SGD (0.01)	79.43	77.41	69.95	79.07	71.46
ReLU	64	SGD (0.01)	80.81	77.56	70.57	78.61	70.43
Logistic regression baseline					77.26	78.16	71.35



(a) Which activation worked best, and why?

Sigmoid came out on top by my selection rule (highest validation macro-F1) and also had the highest test accuracy, 79.67%. But the three activations are really very close. Test accuracy is between 78.6% and 79.7%, and with only 664 test students one standard error is about 1.6 points, so I cannot say

any activation is truly better. Averaged over all 24 runs the validation accuracy is 76.3% (sigmoid), 76.6% (tanh) and 76.2% (ReLU).

The learning curves still show the differences predicted by Q1.3. ReLU reaches the lowest training loss (about 0.47 at epoch 100), because its derivative is 1 for positive inputs and the gradient does not shrink. Sigmoid is the slowest: its best runs needed the largest learning rate (0.1) and, on average, the best epoch was 51 for sigmoid against 23–24 for tanh and ReLU. Its curve is also noisier. This fits the fact that $\sigma'(z) \leq 0.25$. With only one or two hidden layers the vanishing-gradient problem is mild, so it slows sigmoid down but does not break it. All three validation losses flatten out around 0.60–0.62, so the limit seems to come from the data and not from the activation.

(b) Optimizer versus plain SGD

I compared SGD with Adam. Adam converges much faster: its lowest validation loss came after 8.6 epochs on average, against 56.8 for SGD (about 6.6 times fewer epochs). But it did not give a better final result. The average validation accuracy over all runs was 76.3% for Adam and 76.4% for SGD, and the average validation macro-F1 was 68.6% and 67.9%. In fact the best configuration for every activation ended up being SGD, though only by small margins. Adam also depends on the learning rate: with lr 0.01 the 64-unit ReLU network only got 74.7% validation accuracy, because the steps are too big. Overall Adam saves time, not accuracy, on this dataset.

(c) Overfitting

Yes. It is clearest with Adam. For ReLU (64 units, Adam, lr 0.001) training accuracy was 83.0% while validation accuracy was 76.2%, a gap of 6.8 points. The validation loss was lowest at epoch 10 and never improved after that, so early stopping ended the run at epoch 25. Other Adam runs behave the same (best epochs between 1 and 27). The 64-unit SGD runs with lr 0.01 do not overfit: their validation loss is still slowly falling at epoch 97, so they are stopped by the epoch limit and not by convergence.

What I would do: keep early stopping, add L2 weight decay (it is implemented as `--l2` but I did not tune it), use dropout, and reduce the inputs. A lot of the 234 features come from rare category values such as the parents' occupations, which a network can memorize. A smaller hidden layer or more data would also help.

(d) Comparison with logistic regression

Logistic regression on the same split gets 78.16% test accuracy and 71.35% macro-F1. The best network gets 79.67% and 73.09%. The other two networks are at 79.07% and 78.61% accuracy, and the ReLU network has a lower macro-F1 (70.43%). So the differences are about one point or less, which is inside the noise. This tells me the relationship between the features and the outcome is mostly linear, and a nonlinear network adds little. The hard part is the *Enrolled* class: for the baseline its F1 is only 0.47, against 0.81 for Dropout and 0.86 for Graduate. Students who are still enrolled sit between the other two groups and are hard to separate with these features, which is why the macro-F1 is so much lower than the accuracy.

Assumptions

- The 10 integer-coded columns (marital status, application mode, course, attendance, qualifications, nationality, parents' qualifications and occupations) are categories and were one-hot encoded; all others are numeric, including the 0/1 columns.
- The output layer always uses Xavier initialization because it feeds a softmax; hidden layers use He (ReLU) or Xavier (sigmoid, tanh).
- Each configuration was run once (seed 42), and training was capped at 100 epochs.
- The test set was used only for the best configuration of each activation and for the baseline.